

Assignment 1: Autocorrect

Welcome to the first assignment of Course 2. This assignment will give you a chance to brush up on your python and probability skills. In doing so, you will implement an auto-correct system that is very effective and useful.

Important Note on Submission to the AutoGrader

Before submitting your assignment to the AutoGrader, please make sure you are not doing the following:

1. You have not added any *extra* `print` statement(s) in the assignment.
2. You have not added any *extra* code cell(s) in the assignment.
3. You have not changed any of the function parameters.
4. You are not using any global variables inside your graded exercises. Unless specifically instructed to do so, please refrain from it and use the local variables instead.
5. You are not changing the assignment code where it is not required, like creating *extra* variables.

If you do any of the following, you will get something like, `Grader Error: Grader feedback not found` (or similarly unexpected) error upon submitting your assignment. Before asking for help/debugging the errors in your assignment, check for these first. If this is the case, and you don't remember the changes you have made, you can get a fresh copy of the assignment by following these [instructions](#).

Table of Contents

- [0 - Overview](#)
 - [0.1 - Edit Distance](#)
- [1 - Data Preprocessing](#)
 - [Exercise 1 - process_data \(UNQ_C1\)](#)
 - [Exercise 2 - get_count \(UNQ_C2\)](#)
 - [Exercise 3 - get_probs \(UNQ_C3\)](#)
- [2 - String Manipulations](#)
 - [Exercise 4 - delete_letter \(UNQ_C4\)](#)
 - [Exercise 5 - switch_letter \(UNQ_C5\)](#)
 - [Exercise 6 - replace_letter \(UNQ_C6\)](#)
 - [Exercise 7 - insert_letter \(UNQ_C7\)](#)
- [3 - Combining the Edits](#)
 - [3.1 - Edit One Letter](#)
 - [Exercise 8 - edit_one_letter \(UNQ_C8\)](#)
 - [3.2 - Edit Two Letters](#)
 - [Exercise 9 - edit_two_letters \(UNQ_C9\)](#)
 - [3.3 - Suggest Spelling Suggestions](#)
 - [Exercise 10 - get_corrections \(UNQ_C20\)](#)
- [4 - Minimum Edit Distance](#)
 - [4.1 - Dynamic Programming](#)
 - [Exercise 11 - min_edit_distance \(UNQ_C11\)](#)
- [5 - Backtrace \(Optional\)](#)

0 - Overview

You use autocorrect every day on your cell phone and computer. In this assignment, you will explore what really goes on behind the scenes. Of course, the model you are about to implement is not identical to the one used in your phone, but it is still quite good.

By completing this assignment you will learn how to:

- Get a word count given a corpus
- Get a word probability in the corpus

- Manipulate strings
- Filter strings
- Implement Minimum edit distance to compare strings and to help find the optimal path for the edits.
- Understand how dynamic programming works

Similar systems are used everywhere.

- For example, if you type in the word "**I am lerningg**", chances are very high that you meant to write "**learning**", as shown in **Figure 1**.

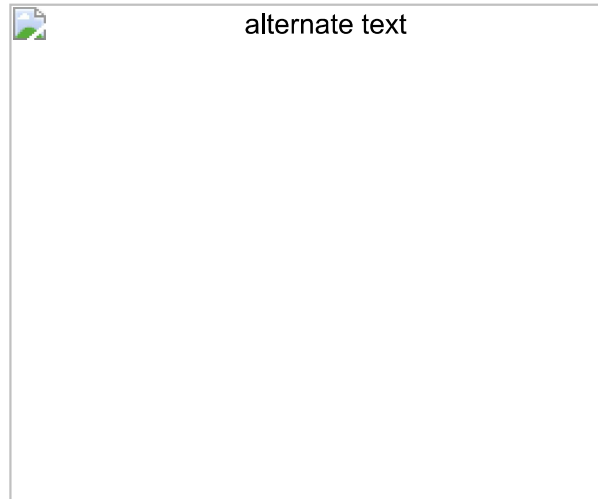


Figure 1

0.1 - Edit Distance

In this assignment, you will implement models that correct words that are 1 and 2 edit distances away.

- We say two words are n edit distance away from each other when we need n edits to change one word into another.

An edit could consist of one of the following options:

- Delete (remove a letter): 'hat' => 'at, ha, ht'
- Switch (swap 2 adjacent letters): 'eta' => 'eat, tea,...'
- Replace (change 1 letter to another): 'jat' => 'hat, rat, cat, mat, ...'
- Insert (add a letter): 'te' => 'the, ten, ate, ...'

You will be using the four methods above to implement an Auto-correct.

- To do so, you will need to compute probabilities that a certain word is correct given an input.

This auto-correct you are about to implement was first created by [Peter Norvig](#) in 2007.

- His [original article](#) may be a useful reference for this assignment.

The goal of our spell check model is to compute the following probability:

$$P(c|w) = \frac{P(w|c) \times P(c)}{P(w)} \quad (\text{Eqn-1})$$

The equation above is [Bayes Rule](#).

- Equation 1 says that the probability of a word being correct $P(c|w)$ is equal to the probability of having a certain word w , given that it is correct $P(w|c)$, multiplied by the probability of being correct in general $P(c)$ divided by the probability of that word w appearing $P(w)$ in general.
- To compute equation 1, you will first import a data set and then create all the probabilities that you need using that data set.

1 - Data Preprocessing

```
import re
from collections import Counter
import numpy as np
import pandas as pd
```

As in any other machine learning task, the first thing you have to do is process your data set.

- Many courses load in pre-processed data for you.
- However, in the real world, when you build these NLP systems, you load the datasets and process them.
- So let's get some real world practice in pre-processing the data!

Your first task is to read in a file called **'shakespeare.txt'** which is found in your file directory. To look at this file you can go to `File ==> Open` .

Exercise 1 - process_data

Implement the function `process_data` which

- 1) Reads in a corpus (text file)
- 2) Changes everything to lowercase
- 3) Returns a list of words.

Options and Hints

- If you would like more of a real-life practice, don't open the 'Hints' below (yet) and try searching the web to derive your answer.
- If you want a little help, click on the green "General Hints" section by clicking on it with your mouse.
- If you get stuck or are not getting the expected results, click on the green 'Detailed Hints' section to get hints for each step that you'll take to complete this function.

General Hints

Detailed Hints

```

import re

# UNQ_C1 GRADED FUNCTION: process_data
def process_data(file_name):
    """
    Input:
        file_name: name of a text file in the current directory
    Output:
        words: a list containing all the words in the corpus in lower case
    """
    words = []

    ### START CODE HERE ###
    # Open the file and read its contents
    with open(file_name, 'r', encoding='utf-8') as file:
        text = file.read()

    # Convert text to Lower case
    text = text.lower()

    # Remove punctuation using regex and split into words
    words = re.findall(r'\b\w+\b', text)
    ### END CODE HERE ###

```

Note, in the following cell, 'words' is converted to a python `set`. This eliminates any duplicate entries.

```

#DO NOT MODIFY THIS CELL
word_1 = process_data('./data/shakespeare.txt')
vocab = set(word_1) # this will be your new vocabulary
print(f"The first ten words in the text are: \n{word_1[0:10]}")

```

The first ten words in the text are:

```
['o', 'for', 'a', 'muse', 'of', 'fire', 'that', 'would', 'ascend', 'the']
```

There are 6116 unique words in the vocabulary.

Expected Output

The first ten words **in** the text are:

```
['o', 'for', 'a', 'muse', 'of', 'fire', 'that', 'would', 'ascend', 'the']
```

There are **6116** unique words **in** the vocabulary.

```
# Test your function
```

```
All tests passed
```

Exercise 2 - get_count

Implement a `get_count` function that returns a dictionary

- The dictionary's keys are words
- The value for each word is the number of times that word appears in the corpus.

For example, given the following sentence: **"I am happy because I am learning"**, your dictionary should return the following:

Key	Value
-----	-------

I	2
am	2
happy	1
because	1
learning	1

Instructions: Implement a `get_count` which returns a dictionary where the key is a word and the value is the number of times the word appears in the list.

Hints

```
# UNIT TEST COMMENT: Candidate for Table Driven Tests
# UNQ_C2 GRADED FUNCTION: get_count
def get_count(word_l):
    """
    Input:
        word_l: a list of words representing the corpus.
    Output:
        word_count_dict: The wordcount dictionary where key is the word and value is its
    """

    word_count_dict = {} # fill this with word counts
    ### START CODE HERE
    for word in word_l:
        if word in word_count_dict:
            word_count_dict[word] += 1
        else:
            word_count_dict[word] = 1
    ### END CODE HERE ###
```

```
#DO NOT MODIFY THIS CELL
word_count_dict = get_count(word_l)
print(f"There are {len(word_count_dict)} key values pairs")
```

There are 6116 key values pairs
The count for the word 'thee' is 240

Expected Output

There are 6116 key values pairs
The count for the word 'thee' is 240

```
# Test your function
```

All tests passed

Exercise 3 - get_probs

Given the dictionary of word counts, compute the probability that each word will appear if randomly selected from the corpus of words.

$$P(w_i) = \frac{C(w_i)}{M} \quad (\text{Eqn-2})$$

where

$C(w_i)$ is the total number of times w_i appears in the corpus.

M is the total number of words in the corpus.

For example, the probability of the word 'am' in the sentence 'I am happy because I am learning' is:

$$P(am) = \frac{C(w_i)}{M} = \frac{2}{7}. \quad (\text{Eqn-3})$$

Instructions: Implement `get_probs` function which gives you the probability that a word occurs in a sample. This returns a dictionary where the keys are words, and the value for each word is its probability in the corpus of words.

Hints

```
# UNQ_C3 GRADED FUNCTION: get_probs
def get_probs(word_count_dict):
    ...

    Input:
        word_count_dict: The wordcount dictionary where key is the word and value is its
    Output:
        probs: A dictionary where keys are the words and the values are the probability t
    ...

    probs = {} # return this variable correctly

    ### START CODE HERE ###
    # Get total count of words
    total_count = sum(word_count_dict.values())

    # Compute probability for each word
    for word, count in word_count_dict.items():
        probs[word] = count / total_count
    ### END CODE HERE ###
```

```
#DO NOT MODIFY THIS CELL
probs = get_probs(word_count_dict)
print(f"Length of probs is {len(probs)}")
```

Length of probs is 6116

P('thee') is 0.0045

Expected Output

Length of probs is 6116

P('thee') is 0.0045

```
# Test your function
```

All tests passed

2 - String Manipulations

Now that you have computed $P(w_i)$ for all the words in the corpus, you will write a few functions to manipulate strings so that you can edit the erroneous strings and return the right spellings of the words. In this section, you will implement four functions:

- `delete_letter` : given a word, it returns all the possible strings that have **one character removed**.

- `switch_letter` : given a word, it returns all the possible strings that have **two adjacent letters switched**.
- `replace_letter` : given a word, it returns all the possible strings that have **one character replaced by another different letter**.
- `insert_letter` : given a word, it returns all the possible strings that have an **additional character inserted**.

List comprehensions

String and list manipulation in python will often make use of a python feature called [list comprehensions](#). The routines below will be described as using list comprehensions, but if you would rather implement them in another way, you are free to do so as long as the result is the same. Further, the following section will provide detailed instructions on how to use list comprehensions and how to implement the desired functions. If you are a python expert, feel free to skip the python hints and move to implementing the routines directly.

Python List Comprehensions embed a looping structure inside of a list declaration, collapsing many lines of code into a single line. If you are not familiar with them, they seem slightly out of order relative to for loops.

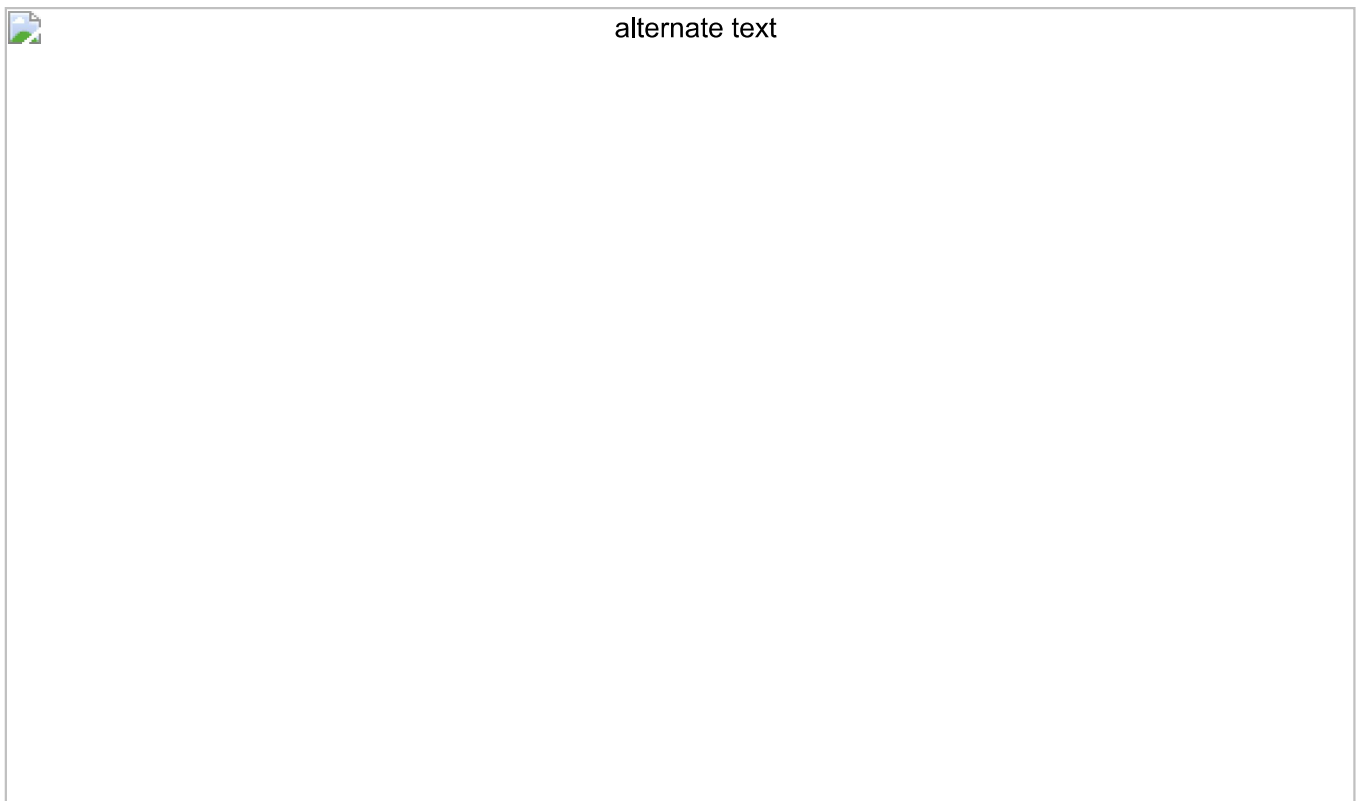


Figure 2

The diagram above shows that the components of a list comprehension are the same components you would find in a typical for loop that appends to a list, but in a different order. With that in mind, we'll continue the specifics of this assignment. We will be very descriptive for the first function, `deletes()`, and less so in later functions as you become familiar with list comprehensions.

Exercise 4 - delete_letter

Instructions for delete_letter(): Implement a `delete_letter()` function that, given a word, returns a list of strings with one character deleted.

For example, given the word **nice**, it would return the set: {'ice', 'nce', 'nic', 'nie'}.

Step 1: Create a list of 'splits'. This is all the ways you can split a word into Left and Right: For example,

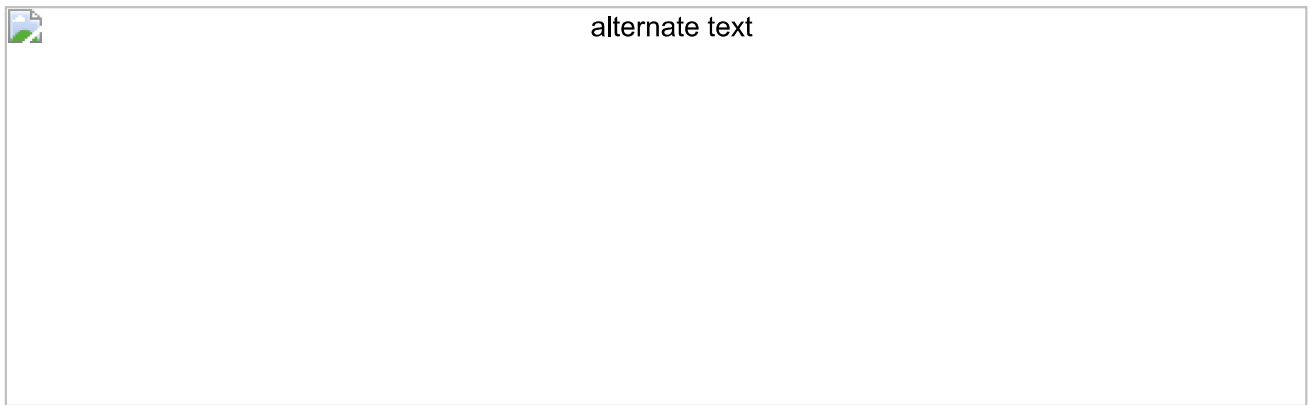


Figure 3

Step 2: This is specific to `delete_letter`. Here, we are generating all words that result from deleting one character.

This can be done in a single line with a list comprehension. You can make use of this type of syntax:

```
[f(a,b) for a, b in splits if condition]
```

For our 'nice' example you get: ['ice', 'nce', 'nie', 'nic']

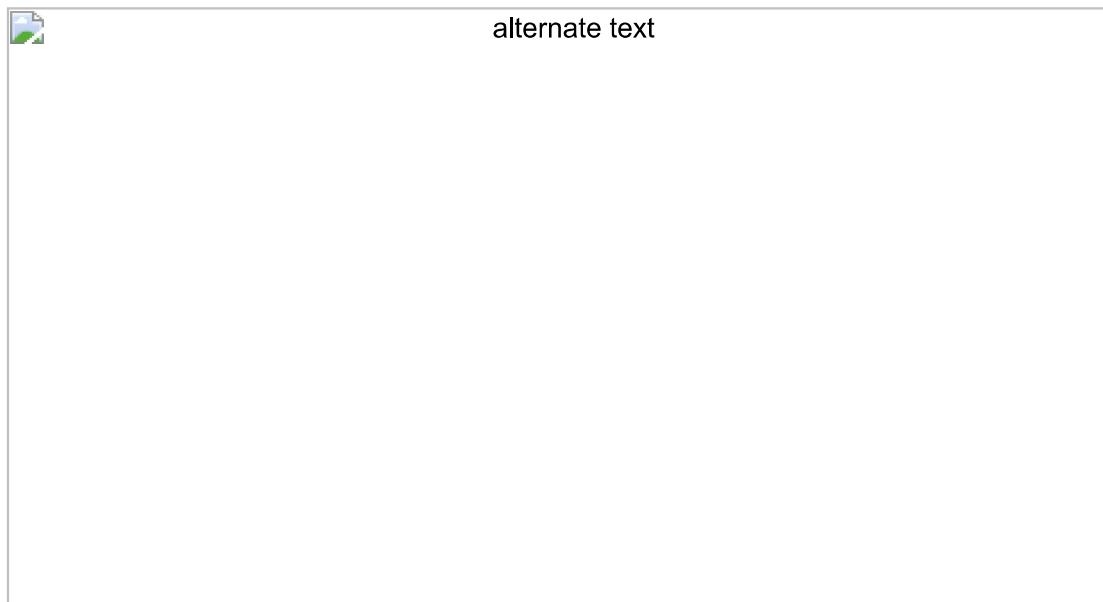


Figure 4

Levels of assistance

Try this exercise with these levels of assistance.

- We hope that this will make it both a meaningful experience but also not a frustrating experience.
- Start with level 1, then move onto level 2, and 3 as needed.
 - Level 1. Try to think this through and implement this yourself.
 - Level 2. Click on the "Level 2 Hints" section for some hints to get started.
 - Level 3. If you would prefer more guidance, please click on the "Level 3 Hints" cell for step by step instructions.
- If you are still stuck, look at the images in the "list comprehensions" section above.

Level 2 Hints

Level 3 Hints


```

# UNIT TEST COMMENT: Candidate for Table Driven Tests
# UNQ_C4 GRADED FUNCTION: delete_letter
def delete_letter(word, verbose=False):
    '''
    Input:
        word: the string/word for which you will generate all possible words
              in the vocabulary which have 1 missing character
    Output:
        delete_l: a list of all possible strings obtained by deleting 1 character from word
    '''

    delete_l = []
    split_l = []

    ### START CODE HERE ###
    # Split the word into all possible (left, right) pairs
    # e.g., for "word": [("", "word"), ("w", "ord"), ("wo", "rd"), ...]
    split_l = [(word[:i], word[i:]) for i in range(len(word))]

    # Generate words by deleting one character at each split
    for L, R in split_l:
        if R: # make sure there is a character to delete
            delete_l.append(L + R[1:])
    ### END CODE HERE ###

    if verbose:
        print(f"input word {word}, \nsplit_l = {split_l}, \ndelete_l = {delete_l}")

```

```

delete_word_l = delete_letter(word="cans",

input word cans,
split_l = [('', 'cans'), ('c', 'ans'), ('ca', 'ns'), ('can', 's')],
delete_l = ['ans', 'cns', 'cas', 'can']

```

Expected Output

Note: You might get a slightly different result with split_l

```

input word cans,
split_l = [('', 'cans'), ('c', 'ans'), ('ca', 'ns'), ('can', 's')],
delete_l = ['ans', 'cns', 'cas', 'can']

```

Note 1

- Notice how it has the extra tuple ('cans', '').
- This will be fine as long as you have checked the size of the right-side substring in tuple (L,R).
- Can you explain why this will give you the same result for the list of deletion strings (delete_l)?

```

input word cans,
split_l = [('', 'cans'), ('c', 'ans'), ('ca', 'ns'), ('can', 's'), ('cans',
'' )],
delete_l = ['ans', 'cns', 'cas', 'can']

```

Note 2

If you end up getting the same word as your input word, like this:

```
input word cans,
split_1 = [('', 'cans'), ('c', 'ans'), ('ca', 'ns'), ('can', 's'), ('cans',
'')],
delete_1 = ['ans', 'cns', 'cas', 'can', 'cans']
```

- Check how you set the `range`.
- See if you check the length of the string on the right-side of the split.

```
# test # 2
```

Number of outputs of `delete_letter('at')` is 2

Expected output

Number of outputs of `delete_letter('at')` is 2

```
# Test your function
```

All tests passed

Exercise 5 - switch_letter

Instructions for switch_letter(): Now implement a function that switches two letters in a word. It takes in a word and returns a list of all the possible switches of two letters **that are adjacent to each other**.

- For example, given the word 'eta', it returns {'eat', 'tea'}, but does not return 'ate'.

Step 1: is the same as in `delete_letter()`

Step 2: A list comprehension or for loop which forms strings by swapping adjacent letters. This is of the form: `[f(L,R) for L, R in splits if condition]` where 'condition' will test the length of R in a given iteration. See below.

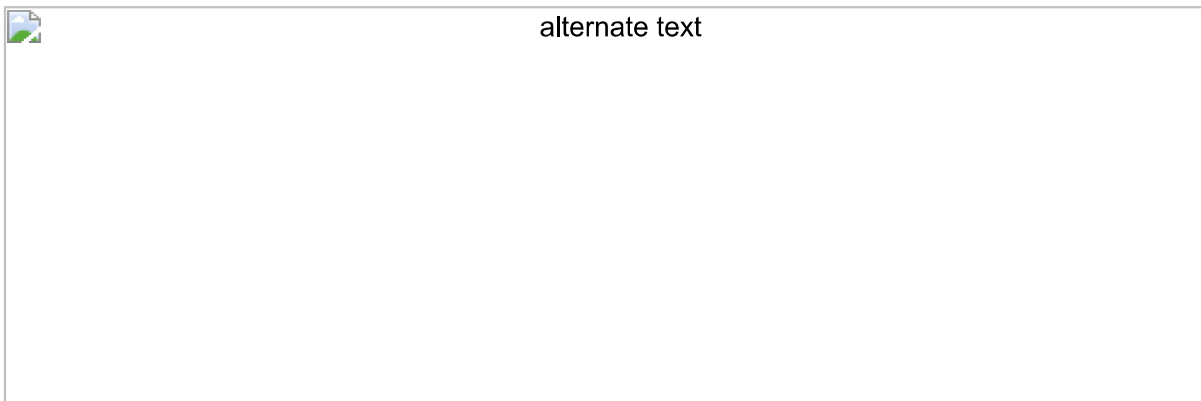


Figure 5

Levels of difficulty

Try this exercise with these levels of difficulty.

- Level 1. Try to think this through and implement this yourself.
- Level 2. Click on the "Level 2 Hints" section for some hints to get started.
- Level 3. If you would prefer more guidance, please click on the "Level 3 Hints" cell for step by step instructions.

Level 2 Hints

Level 3 Hints

```
# UNIT TEST COMMENT: Candidate for Table Driven Tests
# UNQ_C5 GRADED FUNCTION: switches
def switch_letter(word, verbose=False):
    """
    Input:
        word: input string
    Output:
        switch_1: a list of all possible strings with one adjacent character switched
    """

    switch_1 = []
    split_1 = []

    ### START CODE HERE ###
    # Generate all possible splits for switching adjacent letters
    split_1 = [(word[:i], word[i:]) for i in range(len(word) - 1)]

    # For each split, swap the first two letters of the right part
    for L, R in split_1:
        if len(R) >= 2:
            switched = L + R[1] + R[0] + R[2:]
            switch_1.append(switched)
    ### END CODE HERE ###

    if verbose:
        print(f"Input word = {word} \nsplit_1 = {split_1} \nswitch_1 = {switch_1}")
```

```
switch_word_1 = switch_letter(word="eta",
```

```
Input word = eta
split_1 = [('', 'eta'), ('e', 'ta')]
switch_1 = ['tea', 'eat']
```

Expected output

```
Input word = eta
split_1 = [('', 'eta'), ('e', 'ta'), ('et', 'a')]
switch_1 = ['tea', 'eat']
```

Note 1

You may get this:

```
Input word = eta
split_1 = [('', 'eta'), ('e', 'ta'), ('et', 'a'), ('eta', '')]
switch_1 = ['tea', 'eat']
```

- Notice how it has the extra tuple `('eta', '')`.
- This is also correct.
- Can you think of why this is the case?

Note 2

If you get an error

IndexError: string index out of **range**

- Please see if you have checked the length of the strings when switching characters.

```
# test # 2
```

Number of outputs of `switch_letter('at')` is 1

Expected output

Number of outputs of `switch_letter('at')` is **1**

```
# Test your function
```

All tests passed

Exercise 6 - replace_letter

Instructions for `replace_letter()`: Now implement a function that takes in a word and returns a list of strings with one **replaced letter** from the original word.

Step 1: is the same as in `delete_letter()`

Step 2: A list comprehension or for loop which form strings by replacing letters. This can be of the form:
`[f(a,b,c) for a, b in splits if condition for c in string]` Note the use of the second for loop.

It is expected in this routine that one or more of the replacements will include the original word. For example, replacing the first letter of 'ear' with 'e' will return 'ear'.

Step 3: Remove the original input letter from the output.

Hints

```

# UNIT TEST COMMENT: Candidate for Table Driven Tests
# UNQ_C6 GRADED FUNCTION: replace_letter
def replace_letter(word, verbose=False):
    """
    Input:
        word: the input string/word
    Output:
        replace_l: a list of all possible strings where we replaced one letter from the c
    """

    letters = 'abcdefghijklmnopqrstuvwxyz'

    replace_l = []
    split_l = []

    ### START CODE HERE ###
    # Create all splits for replacing each character
    split_l = [(word[:i], word[i:]) for i in range(len(word))]

    # Replace each character with every letter in the alphabet
    for L, R in split_l:
        if R: # make sure there is a character to replace
            for c in letters:
                if c != R[0]: # avoid replacing with the same letter
                    replace_l.append(L + c + R[1:])
    ### END CODE HERE ###

    # Sort the list for easier viewing
    replace_l = sorted(replace_l)

    if verbose:
        print(f"Input word = {word} \nsplit_l = {split_l} \nreplace_l = {replace_l}")

```

```
replace_l = replace_letter(word='can',
```

```

Input word = can
split_l = [('', 'can'), ('c', 'an'), ('ca', 'n')]
replace_l = ['aan', 'ban', 'caa', 'cab', 'cac', 'cad', 'cae', 'caf', 'cag', 'cah', 'cai', 'caj', 'cak', 'cal', 'cam', 'cao', 'cap', 'caq', 'car', 'cas', 'cat', 'cau', 'cav', 'caw', 'cax', 'cay', 'caz', 'cbn', 'ccn', 'cdn', 'cen', 'cfn', 'cgn', 'chn', 'cin', 'cjn', 'ckn', 'cln', 'cmn', 'cnn', 'con', 'cpn', 'cqn', 'crn', 'csn', 'ctn', 'cun', 'cvn', 'cwn', 'cxn', 'cyn', 'czn', 'dan', 'ean', 'fan', 'gan', 'han', 'ian', 'jan', 'kan', 'lan', 'man', 'nan', 'oan', 'pan', 'qan', 'ran', 'san', 'tan', 'uan', 'van', 'wan', 'xan', 'yan', 'zan']

```

Expected Output**:

```

Input word = can
split_l = [('', 'can'), ('c', 'an'), ('ca', 'n')]
replace_l ['aan', 'ban', 'caa', 'cab', 'cac', 'cad', 'cae', 'caf', 'cag', 'cah', 'cai', 'caj', 'cak', 'cal', 'cam', 'cao', 'cap', 'caq', 'car', 'cas', 'cat', 'cau', 'cav', 'caw', 'cax', 'cay', 'caz', 'cbn', 'ccn', 'cdn', 'cen', 'cfn', 'cgn', 'chn', 'cin', 'cjn', 'ckn', 'cln', 'cmn', 'cnn', 'con', 'cpn', 'cqn', 'crn', 'csn', 'ctn', 'cun', 'cvn', 'cwn', 'cxn', 'cyn', 'czn', 'dan', 'ean', 'fan', 'gan', 'han', 'ian', 'jan', 'kan', 'lan', 'man', 'nan', 'oan', 'pan', 'qan', 'ran', 'san', 'tan', 'uan', 'van', 'wan', 'xan', 'yan', 'zan']

```

- Note how the input word 'can' should not be one of the output words.

Note 1

If you get something like this:

```
Input word = can
split_l = [('', 'can'), ('c', 'an'), ('ca', 'n'), ('can', '')]
replace_l ['aan', 'ban', 'caa', 'cab', 'cac', 'cad', 'cae', 'caf', 'cag', 'cah',
'cai', 'caj', 'cak', 'cal', 'cam', 'cao', 'cap', 'caq', 'car', 'cas', 'cat', 'cau',
'cav', 'caw', 'cax', 'cay', 'caz', 'cbn', 'ccn', 'cdn', 'cen', 'cfn', 'cgn',
'chn', 'cin', 'cjn', 'ckn', 'cln', 'cmn', 'cnn', 'con', 'cpn', 'cqn', 'crn', 'csn',
'ctn', 'cun', 'cvn', 'cwn', 'cxn', 'cyn', 'czn', 'dan', 'ean', 'fan', 'gan',
'han', 'ian', 'jan', 'kan', 'lan', 'man', 'nan', 'oan', 'pan', 'qan', 'ran', 'san',
'tan', 'uan', 'van', 'wan', 'xan', 'yan', 'zan']
```

- Notice how split_l has an extra tuple ('can', ''), but the output is still the same, so this is okay.

Note 2

If you get something like this:

```
Input word = can
split_l = [('', 'can'), ('c', 'an'), ('ca', 'n'), ('can', '')]
replace_l ['aan', 'ban', 'caa', 'cab', 'cac', 'cad', 'cae', 'caf', 'cag', 'cah',
'cai', 'caj', 'cak', 'cal', 'cam', 'cana', 'canb', 'canc', 'cand', 'cane', 'canf',
'cang', 'canh', 'cani', 'canj', 'cank', 'canl', 'canm', 'cann', 'cano', 'canp',
'canq', 'canr', 'cans', 'cant', 'canu', 'canv', 'canw', 'canx', 'cany', 'canz',
'cao', 'cap', 'caq', 'car', 'cas', 'cat', 'cau', 'cav', 'caw', 'cax', 'cay',
'caz', 'cbn', 'ccn', 'cdn', 'cen', 'cfn', 'cgn', 'chn', 'cin', 'cjn', 'ckn', 'cln',
'cmn', 'cnn', 'con', 'cpn', 'cqn', 'crn', 'csn', 'ctn', 'cun', 'cvn', 'cwn',
'cxn', 'cyn', 'czn', 'dan', 'ean', 'fan', 'gan', 'han', 'ian', 'jan', 'kan', 'lan',
'man', 'nan', 'oan', 'pan', 'qan', 'ran', 'san', 'tan', 'uan', 'van', 'wan',
'xan', 'yan', 'zan']
```

- Notice how there are strings that are 1 letter longer than the original word, such as cana .
- Please check for the case when there is an empty string '', and if so, do not use that empty string when setting replace_l.

```
# test # 2
```

Number of outputs of replace_letter('at') is 50

Expected output

Number of outputs of replace_letter('at') is 50

```
# Test your function
```

All tests passed

Exercise 7 - insert_letter

Instructions for insert_letter(): Now implement a function that takes in a word and returns a list with a letter inserted at every offset.

Step 1: is the same as in `delete_letter()`

Step 2: This can be a list comprehension of the form:

```
[f(a+b+c) for a, b in splits if condition for c in string]
```

```
# UNIT TEST COMMENT: Candidate for Table Driven Tests
# UNQ_C7 GRADED FUNCTION: inserts
def insert_letter(word, verbose=False):
    """
    Input:
        word: the input string/word
    Output:
        insert_l: a list of all possible strings with one new letter inserted at every of
    """
    letters = 'abcdefghijklmnopqrstuvwxyz'
    insert_l = []
    split_l = []

    ### START CODE HERE ###
    # Generate all splits including the position after the last character
    split_l = [(word[:i], word[i:]) for i in range(len(word) + 1)]

    # Insert each letter of the alphabet at each split
    for L, R in split_l:
        for c in letters:
            insert_l.append(L + c + R)
    ### END CODE HERE ###

    if verbose:
        print(f"Input word {word} \nsplit_l = {split_l} \ninsert_l = {insert_l}")
```

```
insert_l = insert_letter('at', True)
```

```
Input word at
split_l = [('', 'at'), ('a', 't'), ('at', '')]
insert_l = ['aat', 'bat', 'cat', 'dat', 'eat', 'fat', 'gat', 'hat', 'iat', 'jat', 'kat', 'lat', 'mat', 'nat', 'oat', 'pat', 'qat', 'rat', 'sat', 'tat', 'uat', 'vat', 'wat', 'xat', 'yat', 'zat', 'aat', 'abt', 'act', 'adt', 'aet', 'aft', 'agt', 'aht', 'ait', 'ajt', 'akt', 'alt', 'amt', 'ant', 'aot', 'apt', 'aqt', 'art', 'ast', 'att', 'aut', 'avt', 'awt', 'axt', 'ayt', 'azt', 'ata', 'atb', 'atc', 'atd', 'ate', 'atf', 'atg', 'ath', 'ati', 'atj', 'atk', 'atl', 'atm', 'atn', 'ato', 'atp', 'atq', 'atr', 'ats', 'att', 'atu', 'atv', 'atw', 'atx', 'aty', 'atz']
```

Number of strings output by `insert_letter('at')` is 78

Expected output

```
Input word at
split_l = [('', 'at'), ('a', 't'), ('at', '')]
insert_l = ['aat', 'bat', 'cat', 'dat', 'eat', 'fat', 'gat', 'hat', 'iat', 'jat', 'kat', 'lat', 'mat', 'nat', 'oat', 'pat', 'qat', 'rat', 'sat', 'tat', 'uat', 'vat', 'wat', 'xat', 'yat', 'zat', 'aat', 'abt', 'act', 'adt', 'aet', 'aft', 'agt', 'aht', 'ait', 'ajt', 'akt', 'alt', 'amt', 'ant', 'aot', 'apt', 'aqt', 'art', 'ast', 'att', 'aut', 'avt', 'awt', 'axt', 'ayt', 'azt', 'ata', 'atb', 'atc', 'atd', 'ate', 'atf', 'atg', 'ath', 'ati', 'atj', 'atk', 'atl', 'atm', 'atn', 'ato', 'atp', 'atq', 'atr', 'ats', 'att', 'atu', 'atv', 'atw', 'atx', 'aty', 'atz']
Number of strings output by insert_letter('at') is 78
```

Note 1

If you get a `split_l` like this:

```
Input word at
split_l = [('', 'at'), ('a', 't')]
insert_l = ['aat', 'bat', 'cat', 'dat', 'eat', 'fat', 'gat', 'hat', 'iat', 'jat', 'kat', 'lat', 'mat', 'nat', 'oat', 'pat', 'qat', 'rat', 'sat', 'tat', 'uat', 'vat', 'wat', 'xat', 'yat', 'zat', 'aat', 'abt', 'act', 'adt', 'aet', 'aft', 'agt', 'aht', 'ait', 'ajt', 'akt', 'alt', 'amt', 'ant', 'aot', 'apt', 'aqt', 'art', 'ast', 'att', 'aut', 'avt', 'awt', 'axt', 'ayt', 'azt']
Number of strings output by insert_letter('at') is 52
```

- Notice that `split_l` is missing the extra tuple `('at', '')`. For insertion, we actually **WANT** this tuple.
- The function is not creating all the desired output strings.
- Check the range that you use for the for loop.

Note 2

If you see this:

```
Input word at
split_l = [('', 'at'), ('a', 't'), ('at', '')]
insert_l = ['aat', 'bat', 'cat', 'dat', 'eat', 'fat', 'gat', 'hat', 'iat', 'jat', 'kat', 'lat', 'mat', 'nat', 'oat', 'pat', 'qat', 'rat', 'sat', 'tat', 'uat', 'vat', 'wat', 'xat', 'yat', 'zat', 'aat', 'abt', 'act', 'adt', 'aet', 'aft', 'agt', 'aht', 'ait', 'ajt', 'akt', 'alt', 'amt', 'ant', 'aot', 'apt', 'aqt', 'art', 'ast', 'att', 'aut', 'avt', 'awt', 'axt', 'ayt', 'azt']
Number of strings output by insert_letter('at') is 52
```

- Even though you may have fixed the `split_l` so that it contains the tuple `('at', '')`, notice that you're still missing some output strings.
 - Notice that it's missing strings such as `'ata'`, `'atb'`, `'atc'` all the way to `'atz'`.
- To fix this, make sure that when you set `insert_l`, you allow the use of the empty string `''`.

```
# Test your function
```

```
All tests passed
```

3 - Combining the Edits

Now that you have implemented the string manipulations, you will create two functions that, given a string, will return all the possible single and double edits on that string. These will be `edit_one_letter()` and `edit_two_letters()`.

3.1 - Edit One Letter

Exercise 8 - edit_one_letter

Instructions: Implement the `edit_one_letter` function to get all the possible edits that are one edit away from a word. The edits consist of the replace, insert, delete, and optionally the switch operation. You should use the previous functions you have already implemented to complete this function. The 'switch' function is a less common edit function, so its use will be selected by an `"allow_switches"` input argument.

Note that those functions return *lists* while this function should return a *python set*. Utilizing a set eliminates any duplicate entries

Hints

```
# UNIT TEST COMMENT: Candidate for Table Driven Tests
# UNQ_C8 GRADED FUNCTION: edit_one_letter
def edit_one_letter(word, allow_switches=True):
    """
    Input:
        word: the string/word for which we will generate all possible words that are one
        allow_switches: boolean flag to include adjacent character swaps
    Output:
        edit_one_set: a set of words with one possible edit. Please return a set. and not
    """

    edit_one_set = set()

    ### START CODE HERE ###
    # Add all possible deletions
    edit_one_set.update(delete_letter(word))

    # Add all possible insertions
    edit_one_set.update(insert_letter(word))

    # Add all possible replacements
    edit_one_set.update(replace_letter(word))

    # Add all possible switches if allowed
    if allow_switches:
        edit_one_set.update(switch_letter(word))
    ### END CODE HERE ###

    # return as a set
```

```
tmp_word = "at"
tmp_edit_one_set = edit_one_letter(tmp_word)
# turn this into a list to sort it, in order to view it
tmp_edit_one_l = sorted(list(tmp_edit_one_set))

print(f"input word {tmp_word} \nedit_one_l \n{tmp_edit_one_l}\n")
print(f"The type of the returned object should be a set {type(tmp_edit_one_set)}")
```

```
input word at
edit_one_l
['a', 'aa', 'aat', 'ab', 'abt', 'ac', 'act', 'ad', 'adt', 'ae', 'aet', 'a
f', 'aft', 'ag', 'agt', 'ah', 'aht', 'ai', 'ait', 'aj', 'ajt', 'ak', 'ak
t', 'al', 'alt', 'am', 'amt', 'an', 'ant', 'ao', 'aot', 'ap', 'apt', 'aq',
'aqt', 'ar', 'art', 'as', 'ast', 'ata', 'atb', 'atc', 'atd', 'ate', 'atf',
'atg', 'ath', 'ati', 'atj', 'atk', 'atl', 'atm', 'atn', 'ato', 'atp', 'at
q', 'atr', 'ats', 'att', 'atu', 'atv', 'atw', 'atx', 'aty', 'atz', 'au',
'aut', 'av', 'avt', 'aw', 'awt', 'ax', 'axt', 'ay', 'ayt', 'az', 'azt', 'b
at', 'bt', 'cat', 'ct', 'dat', 'dt', 'eat', 'et', 'fat', 'ft', 'gat', 'g
t', 'hat', 'ht', 'iat', 'it', 'jat', 'jt', 'kat', 'kt', 'lat', 'lt', 'ma
t', 'mt', 'nat', 'nt', 'oat', 'ot', 'pat', 'pt', 'qat', 'qt', 'rat', 'rt',
'sat', 'st', 't', 'ta', 'tat', 'tt', 'uat', 'ut', 'vat', 'vt', 'wat', 'w
t', 'xat', 'xt', 'yat', 'yt', 'zat', 'zt']
```

The type of the returned object should be a set <class 'set'>
Number of outputs from edit_one_letter('at') is 129

Expected Output

```
input word at
edit_one_l
['a', 'aa', 'aat', 'ab', 'abt', 'ac', 'act', 'ad', 'adt', 'ae', 'aet', 'af', 'af
t', 'ag', 'agt', 'ah', 'aht', 'ai', 'ait', 'aj', 'ajt', 'ak', 'akt', 'al', 'al
t', 'am', 'amt', 'an', 'ant', 'ao', 'aot', 'ap', 'apt', 'aq', 'aqt', 'ar', 'ar
t', 'as', 'ast', 'ata', 'atb', 'atc', 'atd', 'ate', 'atf', 'atg', 'ath', 'ati',
'atj', 'atk', 'atl', 'atm', 'atn', 'ato', 'atp', 'atq', 'atr', 'ats', 'att', 'at
u', 'atv', 'atw', 'atx', 'aty', 'atz', 'au', 'aut', 'av', 'avt', 'aw', 'awt', 'a
x', 'axt', 'ay', 'ayt', 'az', 'azt', 'bat', 'bt', 'cat', 'ct', 'dat', 'dt', 'ea
t', 'et', 'fat', 'ft', 'gat', 'gt', 'hat', 'ht', 'iat', 'it', 'jat', 'jt', 'ka
t', 'kt', 'lat', 'lt', 'mat', 'mt', 'nat', 'nt', 'oat', 'ot', 'pat', 'pt', 'qa
t', 'qt', 'rat', 'rt', 'sat', 'st', 't', 'ta', 'tat', 'tt', 'uat', 'ut', 'vat',
'vt', 'wat', 'wt', 'xat', 'xt', 'yat', 'yt', 'zat', 'zt']
```

The type of the returned object should be a set `<class 'set'>`
Number of outputs from `edit_one_letter('at')` is 129

```
# Test your function
```

All tests passed

3.2 - Edit Two Letters

Exercise 9 - edit_two_letters

Now you can generalize this to implement to get two edits on a word. To do so, you would have to get all the possible edits on a single word and then for each modified word, you would have to modify it again.

Instructions: Implement the `edit_two_letters` function that returns a set of words that are two edits away. Note that creating additional edits based on the `edit_one_letter` function may 'restore' some one_edits to zero or one edits. That is allowed here. This is accounted for in `get_corrections`.

Hints

```

# UNIT TEST COMMENT: Candidate for Table Driven Tests
# UNQ_C9 GRADED FUNCTION: edit_two_letters
def edit_two_letters(word, allow_switches=True):
    """
    Input:
        word: the input string/word
        allow_switches: boolean flag to include adjacent character swaps
    Output:
        edit_two_set: a set of strings with all possible two edits
    """

    edit_two_set = set()

    ### START CODE HERE ###
    # First, generate all words one edit away
    one_edits = edit_one_letter(word, allow_switches=allow_switches)

    # For each word from one edit, generate another edit
    for w1 in one_edits:
        edit_two_set.update(edit_one_letter(w1, allow_switches=allow_switches))
    ### END CODE HERE ###

    # return as a set

```

```

tmp_edit_two_set = edit_two_letters("a")
tmp_edit_two_l = sorted(list(tmp_edit_two_set))
print(f"Number of strings with edit distance of two: {len(tmp_edit_two_l)}")
print(f"First 10 strings {tmp_edit_two_l[:10]}")
print(f>Last 10 strings {tmp_edit_two_l[-10:]}")
print(f"The data type of the returned object should be a set {type(tmp_edit_two_set)}")

```

Number of strings with edit distance of two: 2654
 First 10 strings ['', 'a', 'aa', 'aaa', 'aab', 'aac', 'aad', 'aae', 'aaf', 'aag']
 Last 10 strings ['zv', 'zva', 'zw', 'zwa', 'zx', 'zxa', 'zy', 'zya', 'zz', 'zza']
 The data type of the returned object should be a set <class 'set'>
 Number of strings that are 2 edit distances from 'at' is 7154

Expected Output

Number of strings with edit distance of two: 2654
 First 10 strings ['', 'a', 'aa', 'aaa', 'aab', 'aac', 'aad', 'aae', 'aaf', 'aag']
 Last 10 strings ['zv', 'zva', 'zw', 'zwa', 'zx', 'zxa', 'zy', 'zya', 'zz', 'zza']
 The data type of the returned object should be a set <class 'set'>
 Number of strings that are 2 edit distances from 'at' is 7154

```
# Test your function
```

All tests passed

3.3 - Suggest Spelling Suggestions

Now you will use your `edit_two_letters` function to get a set of all the possible 2 edits on your word. You will then use those strings to get the most probable word you meant to type a.k.a your typing suggestion.

Exercise 10 - get_corrections

Instructions: Implement `get_corrections`, which returns a list of zero to n possible suggestion tuples of the form (word, probability_of_word).

Step 1: Generate suggestions for a supplied word: You'll use the edit functions you have developed. The 'suggestion algorithm' should follow this logic:

- If the word is in the vocabulary, suggest the word.
- Otherwise, if there are suggestions from `edit_one_letter` that are in the vocabulary, use those.
- Otherwise, if there are suggestions from `edit_two_letters` that are in the vocabulary, use those.
- Otherwise, suggest the input word.*
- The idea is that words generated from fewer edits are more likely than words with more edits.

Note:

- Edits of two letters may 'restore' strings to either zero or one edit. This algorithm accounts for this by preferentially selecting lower distance edits first.

Short circuit

In Python, logical operations such as `and` and `or` have two useful properties. They can operate on lists and they have ['short-circuit' behavior](#). Try these:

```
# example of logical operation on lists or sets
print( [] and ["a","b"] )
print( [] or ["a","b"] )
#example of Short circuit behavior
val1 = ["Most","Likely"] or ["Less","so"] or ["least","of","all"] # selects first, does
print(val1)
val2 = [] or [] or ["least","of","all"] # continues evaluation until there is a non-empty
[]
['a', 'b']
['Most', 'Likely']
['least', 'of', 'all']
```

The logical `or` could be used to implement the suggestion algorithm very compactly. Alternately, `if/elif/else` constructs could be used.

Step 2: Create a 'best_words' dictionary where the 'key' is a suggestion and the 'value' is the probability of that word in your vocabulary. If the word is not in the vocabulary, assign it a probability of 0.

Step 3: Select the n best suggestions. There may be fewer than n.

Hints

```

# UNQ_C10 GRADED FUNCTION: get_corrections
def get_corrections(word, probs, vocab, n=2, verbose=False):
    '''
    Input:
        word: a user entered string to check for suggestions
        probs: a dictionary that maps each word to its probability in the corpus
        vocab: a set containing all the vocabulary
        n: number of possible word corrections you want returned
    Output:
        n_best: a list of tuples with the most probable n corrected words and their probabilities
    '''

    suggestions = set()

    ### START CODE HERE ###
    # Step 1: Generate one-edit candidates that are in the vocabulary
    one_edits = {w for w in edit_one_letter(word) if w in vocab}

    # Step 2: If one-edit candidates exist, use them; otherwise, generate two-edit candidates
    if one_edits:
        candidates = one_edits
    else:
        two_edits = {w for w in edit_two_letters(word) if w in vocab}
        candidates = two_edits

    # Step 3: Include the original word if it is valid
    if word in vocab:
        candidates.add(word)

    suggestions = candidates

    # Step 4: Assign probabilities
    candidates_probs = [(w, probs.get(w, 0)) for w in suggestions]

    # Step 5: Sort by probability descending and select top n
    n_best = sorted(candidates_probs, key=lambda x: x[1], reverse=True)[:n]
    ### END CODE HERE ###

    if verbose:
        print("entered word = ", word, "\nsuggestions = ", suggestions)

```

```

# Test your implementation - feel free to try other words in my word
my_word = 'dys'
tmp_corrections = get_corrections(my_word, probs, vocab, 2, verbose=True) # keep verbose=True
for i, word_prob in enumerate(tmp_corrections):
    print(f"word {i}: {word_prob[0]}, probability {word_prob[1]:.6f}")

```

CODE REVIEW COMMENT: using "tmp_corrections" insteads of "cors". "cors" is not defined

```

entered word = dys
suggestions = {'dye', 'days'}
word 0: days, probability 0.000410
word 1: dye, probability 0.000019
data type of corrections <class 'list'>

```

Expected Output

- Note: This expected output is for `my_word = 'dys'`. Also, keep `verbose=True`

```

entered word = dys
suggestions = {'days', 'dye'}
word 0: days, probability 0.000410
word 1: dye, probability 0.000019
data type of corrections <class 'list'>

```

```
# Test your function
```

```
All tests passed
```

4 - Minimum Edit Distance

Now that you have implemented your auto-correct, how do you evaluate the similarity between two strings? For example: 'waht' and 'what'

Also how do you efficiently find the shortest path to go from the word, 'waht' to the word 'what'?

You will implement a dynamic programming system that will tell you the minimum number of edits required to convert a string into another string.

4.1 - Dynamic Programming

Dynamic Programming breaks a problem down into subproblems which can be combined to form the final solution. Here, given a string `source[0..i]` and a string `target[0..j]`, we will compute all the combinations of substrings `[i, j]` and calculate their edit distance. To do this efficiently, we will use a table to maintain the previously computed substrings and use those to calculate larger substrings.

You have to create a matrix and update each element in the matrix as follows:

Initialization

$$\begin{aligned}
 D[0, 0] &= 0 \\
 D[i, 0] &= D[i - 1, 0] + \text{del_cost}(\text{source}[i]) \\
 D[0, j] &= D[0, j - 1] + \text{ins_cost}(\text{target}[j])
 \end{aligned}
 \tag{4}$$

Per Cell Operations

$$D[i, j] = \min \begin{cases} D[i - 1, j] + \text{del_cost} \\ D[i, j - 1] + \text{ins_cost} \\ D[i - 1, j - 1] + \begin{cases} \text{rep_cost}; & \text{if } \text{src}[i] \neq \text{tar}[j] \\ 0; & \text{if } \text{src}[i] = \text{tar}[j] \end{cases} \end{cases}
 \tag{5}$$

So converting the source word **play** to the target word **stay**, using an input cost of one, a delete cost of 1, and replace cost of 2 would give you the following table:

	#	s	t	a	y
#	0	1	2	3	4
p	1	2	3	4	5
l	2	3	4	5	6
a	3	4	5	4	5

y 4 5 6 5 4

The operations used in this algorithm are 'insert', 'delete', and 'replace'. These correspond to the functions that you defined earlier: insert_letter(), delete_letter() and replace_letter(). switch_letter() is not used here.

The diagram below describes how to initialize the table. Each entry in D[i,j] represents the minimum cost of converting string source[0:i] to string target[0:j]. The first column is initialized to represent the cumulative cost of deleting the source characters to convert string "EER" to "". The first row is initialized to represent the cumulative cost of inserting the target characters to convert from "" to "NEAR".

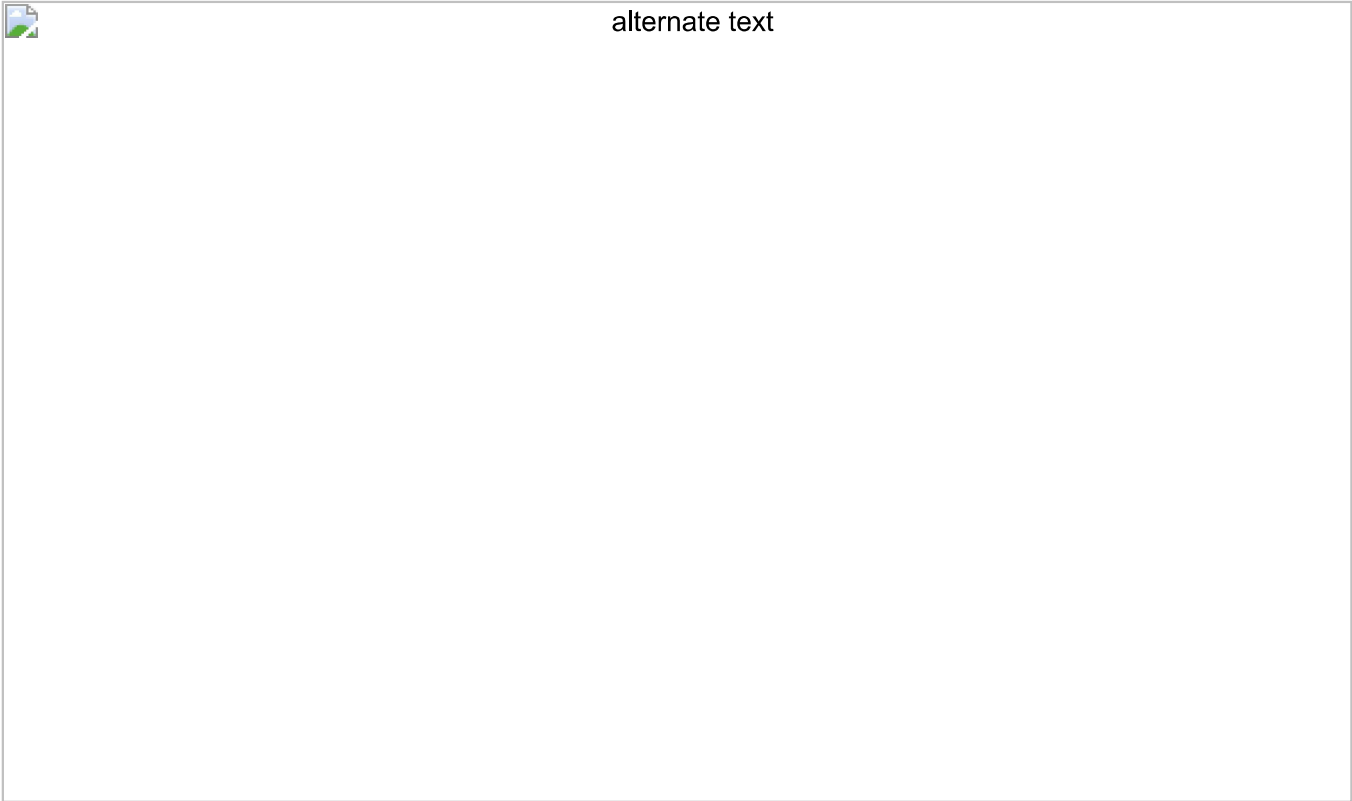


Figure 6 Initializing Distance Matrix

Filling in the remainder of the table utilizes the 'Per Cell Operations' in the equation (5) above. Note, the diagram below includes in the table some of the 3 sub-calculations shown in light grey. Only 'min' of those operations is stored in the table in the min_edit_distance() function.

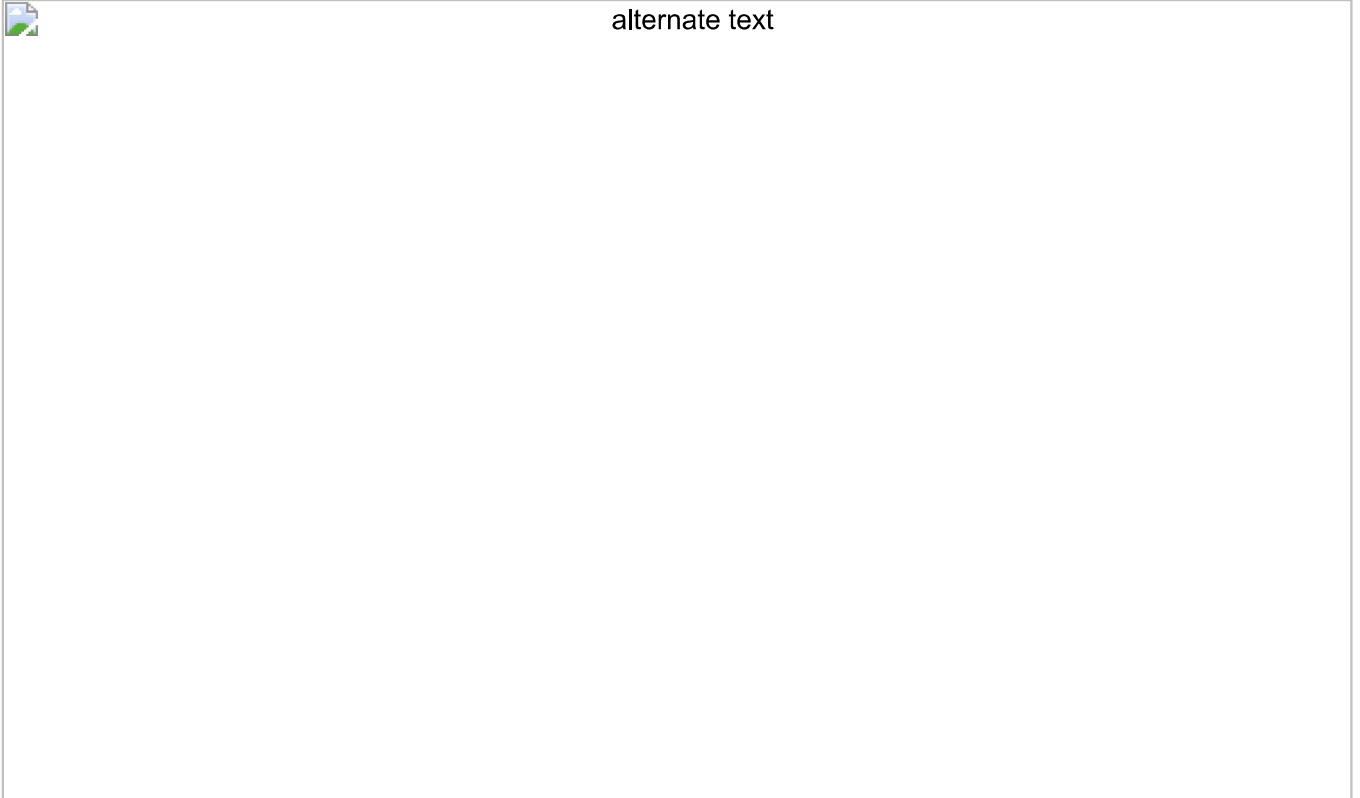


Figure 7 Filling Distance Matrix

Note that the formula for $D[i, j]$ shown in the image is equivalent to:

$$D[i, j] = \min \begin{cases} D[i-1, j] + del_cost \\ D[i, j-1] + ins_cost \\ D[i-1, j-1] + \begin{cases} rep_cost; & if\ src[i] \neq tar[j] \\ 0; & if\ src[i] = tar[j] \end{cases} \end{cases} \quad (5)$$

The variable `sub_cost` (for substitution cost) is the same as `rep_cost` ; replacement cost. We will stick with the term "replace" whenever possible.

Below are some examples of cells where replacement is used. This also shows the minimum path from the lower right final position where "EER" has been replaced by "NEAR" back to the start. This provides a starting point for the optional 'backtrace' algorithm below.

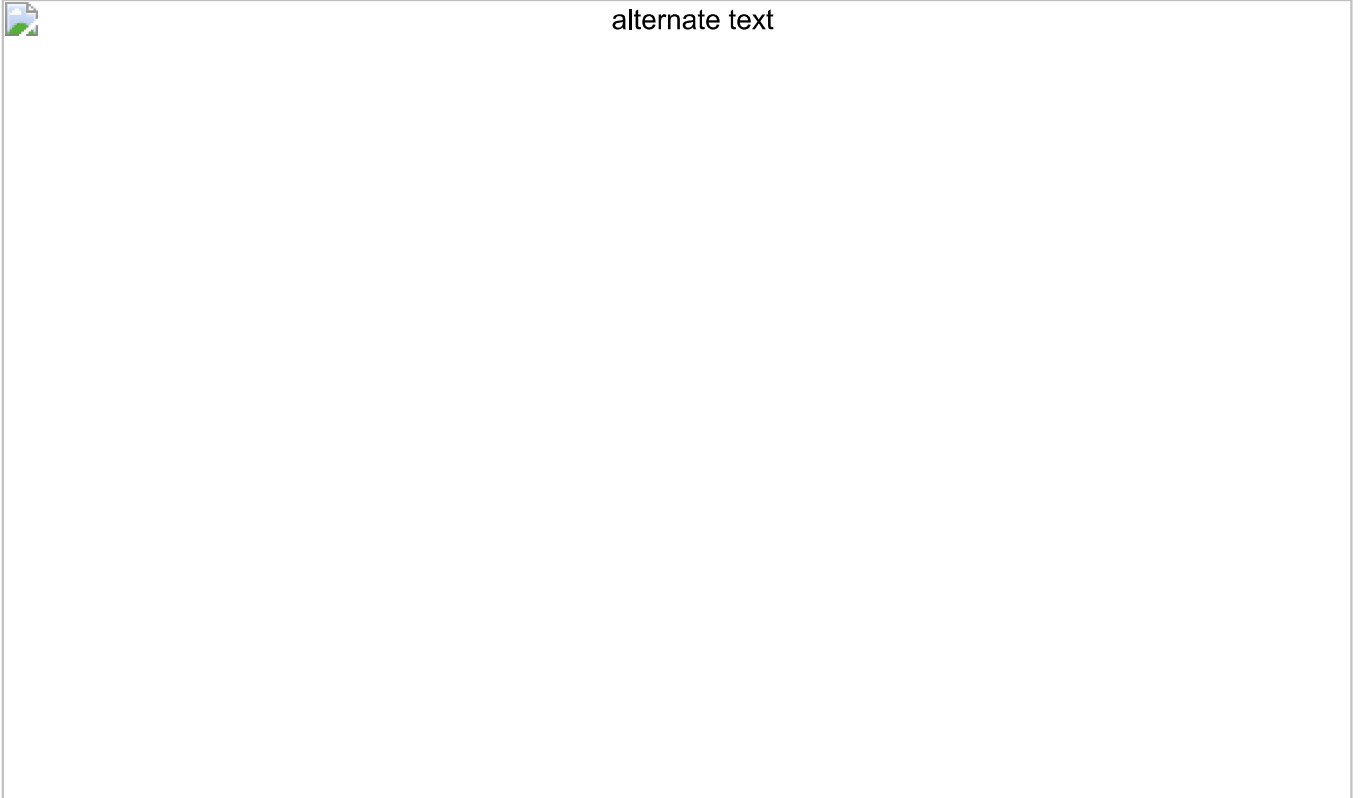


Figure 8 Examples Distance Matrix

Exercise 11 - min_edit_distance

Again, the word "substitution" appears in the figure, but think of this as "replacement".

Instructions: Implement the function below to get the minimum amount of edits required given a source string and a target string.

Hints

```

import numpy as np

# UNQ_C11 GRADED FUNCTION: min_edit_distance
def min_edit_distance(source, target, ins_cost=1, del_cost=1, rep_cost=2):
    """
    Input:
        source: a string corresponding to the string you are starting with
        target: a string corresponding to the string you want to end with
        ins_cost: cost of insertion
        del_cost: cost of deletion
        rep_cost: cost of replacement
    Output:
        D: a matrix containing minimum edit distances
        med: the minimum edit distance to convert source to target
    """
    m = len(source)
    n = len(target)

    # Initialize cost matrix
    D = np.zeros((m+1, n+1), dtype=int)

    ### START CODE ###
    # Initialize first column (deletions)
    for row in range(1, m+1):
        D[row, 0] = row * del_cost

    # Initialize first row (insertions)
    for col in range(1, n+1):
        D[0, col] = col * ins_cost

    # Fill the matrix
    for row in range(1, m+1):
        for col in range(1, n+1):
            # Replacement cost: if same character, cost is 0
            if source[row-1] == target[col-1]:
                r_cost = 0
            else:
                r_cost = rep_cost

            D[row, col] = min(
                D[row-1, col] + del_cost,      # Deletion
                D[row, col-1] + ins_cost,      # Insertion
                D[row-1, col-1] + r_cost        # Replacement / match
            )

    # Minimum edit distance is bottom-right cell
    med = D[m, n]
    ### END CODE ###

```

```
#DO NOT MODIFY THIS CELL
# testing your implementation
source = 'play'
target = 'stay'
matrix, min_edits = min_edit_distance(source, target)
print("minimum edits: ",min_edits, "\n")
idx = list('#' + source)
cols = list('#' + target)
df = pd.DataFrame(matrix, index=idx, columns= cols)
```

minimum edits: 4

	#	s	t	a	y
#	0	1	2	3	4
p	1	2	3	4	5
l	2	3	4	5	6
a	3	4	5	4	5
y	4	5	6	5	4

Expected Results:

minimum edits: 4

	#	s	t	a	y
#	0	1	2	3	4
p	1	2	3	4	5
l	2	3	4	5	6
a	3	4	5	4	5
y	4	5	6	5	4

```
#DO NOT MODIFY THIS CELL
# testing your implementation
source = 'eer'
target = 'near'
matrix, min_edits = min_edit_distance(source, target)
print("minimum edits: ",min_edits, "\n")
idx = list(source)
idx.insert(0, '#')
cols = list(target)
cols.insert(0, '#')
df = pd.DataFrame(matrix, index=idx, columns= cols)
```

minimum edits: 3

	#	n	e	a	r
#	0	1	2	3	4
e	1	2	1	2	3
e	2	3	2	3	4
r	3	4	3	4	3

Expected Results

```
# Test your function
```

```
All tests passed
```

We can now test several of our routines at once:

```
source = "eer"  
targets = edit_one_letter(source, allow_switches = False) #disable switches since min_edit_distance  
for t in targets:  
    _, min_edits = min_edit_distance(source, t, 1, 1, 1) # set ins, del, sub costs all to 1
```

Expected Results

```
(empty)
```

The 'replace()' routine utilizes all letters a-z one of which returns the original word.

```
source = "eer"  
targets = edit_two_letters(source, allow_switches = False) #disable switches since min_edit_distance  
for t in targets:  
    _, min_edits = min_edit_distance(source, t, 1, 1, 1) # set ins, del, sub costs all to 1
```

```
eer eer 0
```

Expected Results

```
eer eer 0
```

We have to allow single edits here because some two_edits will restore a single edit.

Submission

Make sure you submit your assignment before you modify anything below

5 - Backtrace (Optional)

Once you have computed your matrix using minimum edit distance, how would find the shortest path from the top left corner to the bottom right corner?

Note that you could use backtrace algorithm. Try to find the shortest path given the matrix that your `min_edit_distance` function returned.

You can use these [lecture slides on minimum edit distance](#) by Dan Jurafsky to learn about the algorithm for backtrace.

```
# Experiment with back trace - insert your code here
```

References

- Dan Jurafsky - Speech and Language Processing - Textbook
- This auto-correct explanation was first done by Peter Norvig in 2007

Natural Language Processing with Probabilistic Models

- ✓ Lecture notebook: Candidates from edit Lab • 1h
- ✓ Minimum edit distance Video • 3 min
- ✓ Minimum edit distance Reading • 5 min
- ✓ Minimum edit distance algorithm Video • 5 min
- ✓ Minimum edit distance algorithm Reading • 3 min
- ✓ Minimum edit distance algorithm II Video • 4 min
- ✓ Minimum edit distance algorithm II Reading • 5 min
- ✓ Minimum edit distance algorithm III Video • 3 min
- ✓ Minimum edit distance III Reading • 4 min
- ✓ Week Conclusion Video • 51 sec
- ✓ Join the DeepLearning.AI Forum to ask questions, get support, or share amazing ideas! Reading • 2 min
- ✓ Lecture Notes W1 Reading • 1 min
- ✓ Auto-correct and Minimum Edit Distance Practice Assignment • Grade: 90%
- (Optional) Downloading your Notebook, Downloading your Workspace and Refreshing your Workspace Reading • 5 min
- ✓ Autocorrect Programming Assignment • Grade: 90.9%

Programming Assignment: Autocorrect

✓ Passed • 100/110 points

Deadline Pass this assignment by Feb 23, 11:59 PM +06

Launch Notebook

Instructions My submissions

Date	Score	Passed
✓ February 18, 2026 2:53 PM +06	100/110	Yes
Process data	10/10	Show grader output
Get count	10/10	Show grader output
Get probs	0/10	Show grader output
Delete letter	10/10	Show grader output
Switch letter	10/10	Show grader output
Replace letter	10/10	Show grader output
Insert letter	10/10	Show grader output
Edit one letter	10/10	Show grader output
Edit two letters	10/10	Show grader output
Get corrections	10/10	Show grader output
Min edit distance	10/10	Show grader output

Go to next item →



Assignment 1: Autocorrect

Welcome to the first assignment of Course 2. This assignment will give you a chance to brush up on your python and probability skills. In doing so, you will implement an auto-correct system that is very effective and useful.

Important Note on Submission to the AutoGrader

Before submitting your assignment to the AutoGrader, please make sure you are not doing the following:

1. You have not added any *extra* `print` statement(s) in the assignment.
2. You have not added any *extra* code cell(s) in the assignment.
3. You have not changed any of the function parameters.
4. You are not using any global variables inside your graded exercises. Unless specifically instructed to do so, please refrain from it and use the local variables instead.
5. You are not changing the assignment code where it is not required, like creating *extra* variables.

If you do any of the following, you will get something like, `Grader Error: Grader feedback not found` (or similarly unexpected) error upon submitting your assignment. Before asking for help/debugging the errors in your assignment, check for these first. If this is the case, and you don't remember the changes you have made, you can get a fresh copy of the assignment by following these [instructions](#).

Table of Contents

- [0 - Overview](#)
 - [0.1 - Edit Distance](#)
- [1 - Data Preprocessing](#)
 - [Exercise 1 - process data \(UNQ_C1\)](#)
 - [Exercise 2 - get_count \(UNQ_C2\)](#)
 - [Exercise 3 - get_probs \(UNQ_C3\)](#)
- [2 - String Manipulations](#)
 - [Exercise 4 - delete_letter \(UNQ_C4\)](#)
 - [Exercise 5 - switch_letter \(UNQ_C5\)](#)
 - [Exercise 6 - replace_letter \(UNQ_C6\)](#)
 - [Exercise 7 - insert_letter \(UNQ_C7\)](#)
- [3 - Combining the Edits](#)
 - [3.1 - Edit One Letter](#)
 - [Exercise 8 - edit_one_letter \(UNQ_C8\)](#)
 - [3.2 - Edit Two Letters](#)
 - [Exercise 9 - edit_two_letters \(UNQ_C9\)](#)
 - [3.3 - Suggest Spelling Suggestions](#)
 - [Exercise 10 - get_corrections \(UNQ_C20\)](#)
- [4 - Minimum Edit Distance](#)
 - [4.1 - Dynamic Programming](#)
 - [Exercise 11 - min_edit_distance \(UNQ_C11\)](#)
- [5 - Backtrace \(Optional\)](#)



Submit assignment

Grades

Close x

Passed • Grade Received: 90.9090909090909%

Submissions

February 18, 2026 2:53 PM +06 (Highest Grade) v

Process data

10/10 points earned

Get count

10/10 points earned

Get probs

0/10 points earned

Delete letter

10/10 points earned

Switch letter

10/10 points earned

Replace letter

10/10 points earned

Insert letter

10/10 points earned